claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-a261423d5d36e34c0.jsonl`
- SHA-256: `e336ef7ae95f2849333ccc8fe7bd9dac980e6ba15e293c5a54ac50333f65cff7`
- Source modified: `2026-06-25T17:02:10+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T16:56:59.171Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with git diff origin/master...HEAD + Read the files). Be skeptical ? default to isReal=false / adjustedSeverity=INVALID if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [MEDIUM] 1h grace does not prevent racing a live upload from a SECOND concurrent process ? it only narrows the window
Location: backend/providers/gemini.py:44-84 (cleanup_orphaned_files) + backend/main.py:1313-1345 (single-instance lock keyed to db_path)
Detail: The docstring claims the grace 'so we never race a concurrent worker mid-upload' and 'Protects a file a live worker may still be processing.' That guarantee is FALSE in the multi-process case the prompt flags. The single-instance lock at backend/main.py:1319 is keyed to db_path+'.lock', so two instances with different RALLY_APP_HOME/--output-dir run concurrently ? and crucially `backend.worker` (backend/worker/__main__.py) is a SEPARATE entrypoint that runs the same Gemini segmenter pipeline but never acquires that lock at all and never calls the sweep. All of these share ONE Gemini File-API namespace (one GEMINI_API_KEY). So: process A is mid-job, its chunk was uploaded at T (create_time=T). At T+1h+?, that same long job is still running (see worst-case lifetime below) OR a NEW chunk for a different in-flight rally was uploaded by A earlier and is being generate()'d. Process B boots, runs the sweep, sees create_time older than cutoff, and deletes the file OUT FROM UNDER process A. A's next `client.files.get(name=...)` / `generate_content(contents=[video_file, ...])` then fails with a 403/404 ? which analyze_video classifies as 'status check failed' or 'generate failed after retries', dropping that chunk's entire time range from the reel (a silent-partial result the segmenter loudly flags as generate_failed_chunks, but the rallies are still gone). The grace makes this rare, not impossible. The honest framing: this is a best-effort GC that can corrupt a concurrent live job; the docstring overstates the safety.
Suggested fix: (a) Soften the docstring to stop asserting it 'never races' ? state it is a heuristic that NARROWS, not eliminates, the race when multiple processes share one API key. (b) Make the file ownership explicit rather than time-heuristic: tag uploads this process owns (e.g. files.upload display_name/metadata carrying a process/run UUID) and have the sweep delete ONLY files whose display_name marks them as a *previous* run, or skip any file whose name is in a live in-process registry. (c) At minimum, gate the sweep behind an opt-in config flag (default OFF) so a multi-tenant / multi-process deployment doesn't silently enable cross-process deletion. The 1h default should be documented as 'must exceed the max single-call lifetime AND assume no other process shares this key.'

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T16:57:01.941Z)

I'll verify this finding by examining the actual code. Let me start by reading the relevant files and the diff.

3. user (2026-06-25T16:57:05.232Z)

```
f00b71e feat(O1/P11): Gemini remote-file GC ? startup orphan sweep
---BRANCH---
o1-p11/gemini-remote-file-gc
```

4. user (2026-06-25T16:57:05.439Z)

```
 backend/main.py               |  25 ++++++++++
 backend/providers/gemini.py   |  43 ++++++++++++++++++
 tests/test_provider_gemini.py | 104 ++++++++++++++++++++++++++++++++++++++++++++
 3 files changed, 172 insertions(+)
```

5. assistant (2026-06-25T16:57:07.429Z)

Let me read the actual changed code in both files.

6. user (2026-06-25T16:57:08.408Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.pipeline.stitcher.watermark_i18n import localize_watermark
 from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
 from backend.reporting import emit_indexer_report
 from backend.results import Failure  # standardized error/result contract
 from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
         sys.exit(1)


+def _maybe_sweep_gemini_orphans(config) -> None:
+    """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+
 def main():
     parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
     if swept:
         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
```

```
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
     # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
     if args.video_path and not args.server and not args.app:
         print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
 import logging
 import os
 import time
+from datetime import datetime, timedelta, timezone
 from typing import Tuple

 from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
         except Exception:
             pass

+    def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+        """Sweep and delete video files orphaned on the Gemini File API by a previous
+        crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+        leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+        that reach a ``finally``/except).
+
+        Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+        creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+        Never raises; returns the count deleted. Intended to run once at startup.
+
+        Args:
+            grace_sec: only delete files older than this (default 1h). Protects a file a
+                live worker may still be processing.
+        """
+        cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+        deleted = 0
+        try:
+            files = list(self.client.files.list())
+        except Exception as e:
+            logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+            return 0
+        for f in files:
+            try:
+                mime = getattr(f, "mime_type", "") or ""
+                if not mime.startswith("video/"):
+                    continue
+                # f.create_time is an RFC3339 string on the File API.
+                ct = getattr(f, "create_time", None)
+                if ct is None:
+                    # No timestamp ? can't prove it's stale; leave it (safe default).
+                    continue
+                created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+                if created > cutoff:
+                    continue  # too recent ? a live worker may own it
+                self.client.files.delete(name=f.name)
+                deleted += 1
+            except Exception as e:
+                logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+        if deleted:
+            logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+        return deleted
```

```
+
     def analyze_video(self, video_path: str, prompt: str,
                       chunk_duration: float = 0.0,
                       label: str = "") -> Tuple[list, dict]:
```

7. assistant (2026-06-25T16:57:14.695Z)

Now let me look at the surrounding context in both files ? particularly the single-instance lock, the `_safe_delete`/upload flow, and how `analyze_video` handles failures.

8. user (2026-06-25T16:57:14.836Z)

```
1       import json
2       import logging
3       import os
4       import time
5       from datetime import datetime, timedelta, timezone
6       from typing import Tuple
7
8       from google import genai
9       from google.genai import types
10
11      from backend.infrastructure.execution_policy import (
12          ExecutionPolicy,
13          PolicyTimeout,
14          poll_until,
15          run_bounded,
16      )
17      from backend.providers.base import AIVideoProvider
18
19      logger = logging.getLogger(__name__)
20
21      # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22      # each uploaded clip is below this. Oversize uploads are refused (not silently
attempted).
23      MAX_UPLOAD_MB = 500
24
25      class GeminiProvider(AIVideoProvider):
26          def __init__(self, api_key: str, model_name: str):
27              super().__init__(api_key, model_name)
28              # Initialize client lazily to avoid connection/auth issues during class
instantiation
29              self._client = None
30
31          @property
32          def client(self):
33              if self._client is None:
34                  self._client = genai.Client(api_key=self.api_key)
35              return self._client
36
37          def _safe_delete(self, video_file) -> None:
38              """Best-effort cleanup of an uploaded File (never raises)."""
39              try:
40                  self.client.files.delete(name=video_file.name)
41              except Exception:
42                  pass
43
44          def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
45              """Sweep and delete video files orphaned on the Gemini File API by a previous
46              crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
47              leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
48              that reach a ``finally``/except).
49
50              Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
whose
```

```
            creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
upload).
            Never raises; returns the count deleted. Intended to run once at startup.

            Args:
                grace_sec: only delete files older than this (default 1h). Protects a file a
                    live worker may still be processing.
            """
            cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
            deleted = 0
            try:
                files = list(self.client.files.list())
            except Exception as e:
                logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
                return 0
            for f in files:
                try:
                    mime = getattr(f, "mime_type", "") or ""
                    if not mime.startswith("video/"):
                        continue
                    # f.create_time is an RFC3339 string on the File API.
                    ct = getattr(f, "create_time", None)
                    if ct is None:
                        # No timestamp ? can't prove it's stale; leave it (safe default).
                        continue
                    created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
                    if created > cutoff:
                        continue  # too recent ? a live worker may own it
                    self.client.files.delete(name=f.name)
                    deleted += 1
                except Exception as e:
                    logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
            if deleted:
                logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
            return deleted

        def analyze_video(self, video_path: str, prompt: str,
                          chunk_duration: float = 0.0,
                          label: str = "") -> Tuple[list, dict]:
            log_prefix = f"Gemini {label}" if label else "Gemini"

            metrics: dict = {
                "upload_time": 0.0,
                "process_time": 0.0,
                "gen_time": 0.0,
            }

            # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
            try:
                size_mb = os.path.getsize(video_path) / (1024 * 1024)
            except OSError:
                size_mb = 0.0
            if size_mb > MAX_UPLOAD_MB:
                logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
                             f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
                             f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
                metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
                metrics["oversize_skipped"] = True
                return [], metrics

```

```
            # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset connections)
            logger.info(f"[{log_prefix}] Uploading {video_path}...")
            t_upload_start = time.time()
            video_file = None
            for attempt in range(3):
                try:
                    video_file = self.client.files.upload(file=video_path)
                    metrics["upload_time"] = time.time() - t_upload_start
                    logger.info(f"[{log_prefix}] Upload complete in {metrics['upload_time']:.1f}s. URI: {video_file.uri}")
                    break
                except Exception as e:
                    logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed: {e}")
                    if attempt < 2:
                        time.sleep(2 * (attempt + 1))
            if video_file is None:
                logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
                metrics["error"] = "upload failed after 3 attempts"
                return [], metrics

            # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must not hang
            # the worker forever). Per-poll logs at DEBUG (no INFO spam); see EXECUTION_POLICY.md.
            t_process_start = time.time()
            process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0)  # 20 min cap

            def _processed():
                nonlocal video_file
                video_file = self.client.files.get(name=video_file.name)
                return None if video_file.state.name == "PROCESSING" else video_file

            try:
                video_file = poll_until(_processed, process_policy,
                                        label=f"{log_prefix} processing", logger=logger)
            except PolicyTimeout:
                logger.error(f"[{log_prefix}] Processing exceeded {process_policy.max_wait_sec:.0f}s ? giving up.")
                self._safe_delete(video_file)
                metrics["error"] = f"processing timed out after {process_policy.max_wait_sec:.0f}s"
                return [], metrics
            except Exception as e:
                logger.error(f"[{log_prefix}] Status check failed: {e}")
                self._safe_delete(video_file)
                metrics["error"] = f"status check failed: {e}"
                return [], metrics
            if video_file.state.name == "FAILED":
                logger.error(f"[{log_prefix}] Video processing failed: {video_file.error.message}")
                self._safe_delete(video_file)
                metrics["error"] = f"video processing failed: {video_file.error.message}"
                return [], metrics

            metrics["process_time"] = time.time() - t_process_start
            logger.info(f"[{log_prefix}] Processing finished in {metrics['process_time']:.1f}s.")

            # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-killer that was
            # missing: a generate() that never returns previously blocked forever) + retry the 503/network
```

```
163             # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
164             logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
165             t_gen_start = time.time()
166             gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167             raw_text = None
168
169             def _generate():
170                 return self.client.models.generate_content(
171                     model=self.model_name,
172                     contents=[video_file, prompt],
173                     config=types.GenerateContentConfig(
174                         response_mime_type="application/json", temperature=0.0),
175                 )
176
177             try:
178                 response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
179                 raw_text = response.text
180                 metrics["gen_time"] = time.time() - t_gen_start
181                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182             except Exception as e:                  # PolicyTimeout (hang) or the last 503 ?
both end here
183                 logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184                 metrics["error"] = f"generate failed after retries: {e}"
185                 metrics["generate_failed"] = True
186             self._safe_delete(video_file)
187
188             if not raw_text:
189                 return [], metrics
190
191             # 4. Parse JSON
192             try:
193                 segments = json.loads(raw_text)
194                 if not isinstance(segments, list):
195                     logger.error(f"[{log_prefix}] Response is not a JSON array.")
196                     metrics["error"] = "response is not a JSON array"
197                     return [], metrics
198                 return segments, metrics
199             except Exception as e:
200                 logger.error(f"[{log_prefix}] JSON parse failed: {e}")
201                 metrics["error"] = f"JSON parse failed: {e}"
202                 return [], metrics
203
```

9. assistant (2026-06-25T16:57:17.454Z)

Now let me examine the single-instance lock in main.py and check whether `backend.worker` is a separate entrypoint that bypasses the lock and sweep.

10. user (2026-06-25T16:57:18.375Z)

```
1300        global_config = load_app_config()  # returns typed AppConfig
1301        _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE
profile
1302
1303        # The CLI --output-dir overrides the configured output directory. It now lives
1304        # on the typed model (OutputConfig.output_dir), so every consumer ? including
1305        # /api/compile ? reads the same value instead of a write-only runtime hack.
1306        # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1307        # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
```

```
        global_config.output.output_dir = resolve_output_dir(args.output_dir)
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)

        # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
        # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
        # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
        # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
        global _instance_lock
        try:
            _instance_lock = acquire_lock(db_path + ".lock")
        except OSError as exc:
            # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
            # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
            # legitimate single instance).
            logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
                           "WITHOUT the second-instance guard.", db_path, exc)
            _instance_lock = None
        else:
            if _instance_lock is None:
                print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
                      f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
                      f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
                sys.exit(1)

        db_instance = Database(db_path)

        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
        # the executor is in-process. Mark them failed so pollers see a terminal state.
        swept = db_instance.fail_stale_jobs()
        if swept:
            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")

        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
        _maybe_sweep_gemini_orphans(global_config)

        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
        if args.video_path and not args.server and not args.app:
            print(f"[CLI] Processing video file: {args.video_path}")
            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
            storage_cfg = getattr(global_config, "storage", None)
```

```
1354            try:
1355                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1356            except ValueError as e:
1357                print(f"[FAIL] {e}")
1358                return
1359            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
```

11. assistant (2026-06-25T16:57:22.446Z)

The lock keying claim is confirmed. Now let me verify the claim that `backend.worker` is a separate entrypoint that runs the Gemini segmenter but never acquires the lock or calls the sweep.

12. user (2026-06-25T16:57:24.130Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__init__.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/worker/__main__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/worker/__pycache__/__init__.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/worker/__pycache__/__main__.cpython-313.pyc
---
__init__.py
__main__.py
__pycache__
```

13. user (2026-06-25T16:57:24.257Z)

```
backend\worker\__init__.py
backend\worker\__main__.py
backend\worker\__pycache__\__init__.cpython-313.pyc
backend\worker\__pycache__\__main__.cpython-313.pyc
```

14. user (2026-06-25T16:57:29.904Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import logging
22      import os
23      import re
24      import sys
25      import tempfile
```

```
    from typing import Any, List

    from backend import storage
    from backend.compute import ComputeJobSpec, get_compute_backend
    from backend.config import (
        StartupCheckError,
        enforce_startup_checks,
        load_config,
        probe_cuda_available,
    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("backend.worker")


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


```

```
88      def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
89          with open(path, "w") as f:
90              json.dump({
91                  "video_id": video_id,
92                  "status": status,
93                  "segment_count": len(segments),
94                  "segments": [{"start": float(s.get("start_time", 0.0)),
95                                "end": float(s.get("end_time", 0.0))} for s in segments],
96              }, f, indent=2)
97
98
99      def _run_startup_checks(config: Any) -> bool:
100         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
101         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
102         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
103         (returns True, ZERO work) when no deployment.profile is selected.
104
105         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
106         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
107         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
108         no-op of work, not just of output."""
109         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
110         cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF
path
111         try:
112             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
113             return True
114         except StartupCheckError:
115             return False  # already logged at ERROR by enforce_startup_checks
116
117
118     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
119         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
120         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
121         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
122
123         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
124         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
125         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
126         from backend.infrastructure.execution_policy import PolicyTimeout
127
128         spec = ComputeJobSpec(
129             video_uri=args.video_uri, out_uri=args.out_uri,
130             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
131         )
132         try:
133             result = compute.run_infer(spec)
134         except PolicyTimeout as e:
135             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
136             return 4
137         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
```

```
                    result.video_id, result.returncode, result.outputs_uri)
        return result.returncode


    def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
                           status: str, storage_cfg: dict, scratch: str) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {"video_id": video_id, "returncode": returncode,
                      "segment_count": segment_count, "status": status}
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(video_id, local_video, "failed" if failures else "processed",
                     [r.model_dump() for r in val_results])

```

```
        def _fail(rc: int) -> int:
            # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
            # dispatcher's bucket-poll terminates FAST + accurately instead of hanging until
its poll
            # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(args.done_uri, video_id, rc, 0, "failed", storage_cfg,
scratch)
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print("[worker] validation FAILED: "
                      + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[worker] unknown segmenter: {seg_name!r} "
                      f"(have {list(segmenter_registry.list_available())})")
                return _fail(2)
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
                                 or getattr(config.indexing, "detector_impl", None) or
"auto")
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
                    "the WASB env produced no detections, or the input resolution differs
from the tuned "
                    "proxy resolution. Re-run with -v for the native command + frame
counts.",
                    video_id, segmenter_actual, detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
```

```
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

    #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")


    def _label_clip_name(fname: str) -> str:
        """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.

        Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
        canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

        * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
        * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
        * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
```

```
        * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)

        Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
        video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
        """
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        return _LABEL_DATE_PREFIX.sub("", name)


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
here we
        probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
        rather than guesses.
        """
        import json
        import subprocess
        try:
            out = subprocess.check_output(
                [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
                 "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
                stderr=subprocess.DEVNULL).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(f"[worker] unknown segmenter: {args.segmenter!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return None
```

```
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                  f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
            return None
        try:
            runner, _ = seg._resolve_runner(config.indexing)
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
        labels_dir = os.path.join(scratch, "labels")
        traj_dir = os.path.join(scratch, "trajectories")
        videos_dir = os.path.join(scratch, "videos")
        os.makedirs(labels_dir, exist_ok=True)
        os.makedirs(traj_dir, exist_ok=True)

        corpus = args.corpus_uri.rstrip("/")
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
```

```
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
            name = _label_clip_name(fname)                         # -> GX020094 (date
prefix stripped)
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
```

```
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(prog="python -m backend.worker",
                                    description="Storage-aware worker: pull ? run pipeline ?
push.")
        p.add_argument("-v", "--verbose", action="store_true",
                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
        pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
        pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
        pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
        pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument("--done-uri", default=None,
                        help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
                             "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
                             "Default-OFF: omit it for the normal local run.")
        pi.add_argument("--set", action="append", default=[], metavar="k=v",
                        help="config override, e.g. --set indexing.skip_ai_handoff=true")
        pi.set_defaults(func=cmd_infer)

```

```
        pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
        pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
                        "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
        pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
                        help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
                             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
                             "(needs videos/ in the corpus; honours detector_impl).")
        pt.add_argument("--segmenter", default="wasb_hybrid",
                        help="trajectory hybrid whose detector is used for --trajectory-
source detector")
        pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
        pt.add_argument("--config", default=None)
        pt.add_argument("--scratch", default=None)
        pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
        pt.add_argument("--frame-width", type=float, default=1280.0,
                        help="synth trajectory frame width (detector probes the video)")
        pt.add_argument("--set", action="append", default=[], metavar="k=v")
        pt.set_defaults(func=cmd_train)
        return p


    def _setup_logging(verbose: bool) -> None:
        """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
        VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
        (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
        That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
        entry), not at import, so importing the module never reconfigures a caller's
logging."""
        # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
        # configured logging at import), which would leave the level at WARNING and re-
suppress the
        # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
        logging.basicConfig(
            level=logging.DEBUG if verbose else logging.INFO,
            format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
            datefmt="%H:%M:%S",
            force=True,
        )


    def main(argv=None) -> int:
        args = build_parser().parse_args(argv)
        _setup_logging(getattr(args, "verbose", False))
        return args.func(args)


    if __name__ == "__main__":
        sys.exit(main())

```

15. assistant (2026-06-25T16:57:35.729Z)

Now I have a clear picture. The worker is a real separate entrypoint. But let me check a critical detail: does the worker actually run the **Gemini** segmenter pipeline that uploads to the File API? The worker uses the segmenter registry. Let me check whether the Gemini provider/segmenter is reachable through the worker's `cmd_infer` path, and whether the worker default-OFF path uses `skip_ai_handoff`.

The key question for the finding's validity: the finding claims `backend.worker` "runs the same Gemini segmenter pipeline." Let me verify the AI handoff path in segmenters actually invokes the Gemini provider's `analyze_video`/`files.upload`.

16. user (2026-06-25T17:00:45.494Z)

```
Found 13 files
backend\main.py
backend\providers\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\eval\gemini_refine.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\worker\__main__.py
backend\providers\__init__.py
backend\providers\base.py
backend\pipeline\segmenters\gemini.py
backend\config\presets.py
backend\config\models.py
backend\config\loader.py
backend\reporting\harvest.py
```

17. user (2026-06-25T17:00:50.257Z)

```
backend\pipeline\segmenters\gemini.py:93:        provider_name = self.config.ai_provider
backend\pipeline\segmenters\gemini.py:108:             provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend\pipeline\segmenters\gemini.py:222:             chunk_segments, metrics =
provider.analyze_video(
backend\pipeline\segmenters\gemini.py:352:         ai_segments, metrics = provider.analyze_video(
backend\pipeline\segmenters\trajectory_hybrid.py:191:        skip_ai =
bool(idx_cfg.skip_ai_handoff)
backend\pipeline\segmenters\trajectory_hybrid.py:192:        provider_name =
self.config.ai_provider
backend\pipeline\segmenters\trajectory_hybrid.py:196:                "%s in FULLY-LOCAL mode
(skip_ai_handoff=True): candidate windows will "
backend\pipeline\segmenters\trajectory_hybrid.py:215:                provider =
provider_registry.get(
backend\pipeline\segmenters\trajectory_hybrid.py:398:                segments, metrics =
provider.analyze_video(
backend\pipeline\segmenters\yolo_hybrid.py:74:        provider_name = self.config.ai_provider
backend\pipeline\segmenters\yolo_hybrid.py:90:            provider =
provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
backend\pipeline\segmenters\yolo_hybrid.py:285:             segments, metrics =
provider.analyze_video(w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi+1}")
backend\pipeline\segmenters\yolo_hybrid.py:382:         provider_name = self.config.ai_provider
```

18. assistant (2026-06-25T17:00:55.754Z)

Confirmed: the trajectory_hybrid and gemini segmenters call `provider.analyze_video` which does `files.upload`. The worker's `cmd_infer` runs these segmenters through the registry (line 216: `segmenter_cls(db, config).process_video(...)`). So the worker CAN upload to the Gemini File API and does NOT acquire the lock and does NOT call the sweep. The finding's structural claims are accurate.

Now let me verify the worst-case lifetime claim to assess whether a file can stay live beyond the 1h grace. The grace is 3600s. Let me check the per-call max lifetime: upload poll (1200s) +

generate (op_timeout 180s × 5 retries + backoffs). Let me also check `run_bounded`/retry math.

19. user (2026-06-25T17:00:57.460Z)

```
4:- ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) + retry-
with-backoff
8:  ``max_wait_sec`` deadline).
10:**Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-tested
with zero real
40:    """An op hung past ``op_timeout_sec`` or never became ready within ``max_wait_sec``."""
48:    op_timeout_sec: float = 120.0         # HARD per-attempt deadline ? the hang-killer that
was missing
49:    max_wait_sec: float = 1800.0          # total wall budget for a poll loop
50:    max_retries: int = 5                  # retries (=> max_retries+1 attempts) for
failures/hangs
51:    backoff_base_sec: float = 3.0         # exponential: backoff_base * 2**attempt
75:    return policy.backoff_base_sec * (2 ** attempt)
78:def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
80:                sleep: Callable[[float], None] = _time.sleep) -> T:
87:    attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
104:        if done.wait(timeout=policy.op_timeout_sec):
113:            last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
115:                          policy.op_timeout_sec)
119:            sleep(wait)
125:def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy = DEFAULT_POLICY,
*,
127:               sleep: Callable[[float], None] = _time.sleep,
130:    ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
138:             label, policy.poll_every_n_sec, policy.max_wait_sec)
145:        if elapsed >= policy.max_wait_sec:
147:            raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
150:        sleep(policy.poll_every_n_sec)
```

20. assistant (2026-06-25T17:01:07.944Z)

Now I can compute the worst-case single-call lifetime for one uploaded file:
- Processing poll: `max_wait_sec=1200.0` (20 min)
- Generate: `op_timeout_sec=180.0`, `max_retries=5` ? 6 attempts × 180s = 1080s, plus backoff sleeps (3·(2^0..2^4) = 3+6+12+24+48 = 93s) ? ~1173s (~20 min)

Total worst case per single chunk: ~1200 + ~1173 ? ~2400s ? ~40 min. That is **under** the 3600s (1h) grace. So for a single chunk, the file is deleted within ~40 min of its create_time in the worst case, comfortably under 1h.

But the finding's stronger point is about **multiple chunks in one long job**: chunk A uploaded at T, processed and deleted; but the whole job has many chunks/windows processed sequentially. A file uploaded early (chunk #1 at create_time=T) is deleted after its own ~40min ? but actually it's deleted right after that chunk's generate completes (`_safe_delete` at line 186). So each individual file's lifetime is bounded by one chunk's ~40min processing, not the whole job. The grace of 1h exceeds that per-chunk worst case. Let me verify there's no path where a file stays alive across multiple chunks.

Looking at the code: each `analyze_video` call uploads ONE file and deletes it at the end of that same call (line 186, or earlier on error paths). So a file's max lifetime on the server during normal operation is one `analyze_video` invocation ? ~40min worst case < 1h grace. The finding's "worst-case lifetime" hand-wave ("see worst-case lifetime below" ? there is no actual figure) is the weak link, but let me confirm there's no chunk-level concurrency or held files.

21. user (2026-06-25T17:01:08.970Z)

```
380                 )
```

```
381                 os.makedirs(handoff_dir, exist_ok=True)
382                 logger.info(
383                     "Phase 3: %s validation on %d candidate windows...",
384                     provider_name,
385                     len(handoff_indices),
386                 )
387
388                 def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
389                     w_start = max(0, window["start"] - handoff_padding)
390                     w_end = min(video_duration, window["end"] + handoff_padding)
391                     w_dur = w_end - w_start
392                     w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
393                     if not extract_video_chunk(
394                         video_path, w_start, w_dur, w_path, reencode=True
395                     ):
396                         return []
397                     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
398                     segments, metrics = provider.analyze_video(
399                         w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
400                     )
401                     # B2: a provider failure returns [] with metrics["error"] set ? that is
402                     # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
403                     # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
404                     if metrics.get("error"):
405                         logger.warning(
406                             "[Handoff %d] provider error ? window %d skipped: %s",
407                             wi + 1,
408                             wi,
409                             metrics["error"],
410                         )
411                     valid = []
412                     for seg in segments:
413                         try:
414                             seg["start_time"] = float(seg["start_time"]) + w_start
```

22. assistant (2026-06-25T17:01:11.284Z)

Let me check whether these candidate windows are processed concurrently (parallel uploads), which would matter for the race window analysis.

23. user (2026-06-25T17:01:12.653Z)

```
21:import concurrent.futures
253:        ai_max_workers = idx_cfg.ai_max_workers
388:            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
430:            with concurrent.futures.ThreadPoolExecutor(
431:                max_workers=ai_max_workers
432:            ) as executor:
434:                    executor.submit(
435:                        process_candidate_window, wi, all_candidate_windows[wi]
```

24. assistant (2026-06-25T17:01:26.674Z)

So within a single job, multiple windows upload concurrently via a ThreadPool, but each file still gets uploaded and deleted within its own `analyze_video` call (~40min worst case). The per-file lifetime is bounded by one `analyze_video` call, not the whole job. This means within a SINGLE process, the 1h grace comfortably exceeds the per-file worst-case lifetime ? so a process never races its OWN in-flight files (the grace is generous relative to per-call lifetime).

The finding's real claim, though, is about the **multi-process / multi-key-shared** scenario: process B's startup sweep deleting process A's live file. This is genuinely possible because:
1. The lock is keyed to `db_path` ? confirmed at main.py:1319.

2. `backend.worker` never acquires the lock and never calls the sweep ? confirmed.
3. Both share one GEMINI_API_KEY namespace (`files.list()` returns all files for the key).
4. A's file at create_time=T can still be live (uploaded, mid-generate within its ~40min call) when B boots and sweeps. For B to delete A's file, the file must be **older than 1h** at B's sweep time. But A only keeps a file alive for ~40min max. So B would only delete a file that A uploaded >1h ago ? which A would have already deleted (since A deletes within ~40min).

This is the crux. Let me think carefully. For B's sweep to delete a file A still owns, the file's create_time must be > 1h old AND A must still be using it. But A's per-call lifetime is ~40min worst case, so A would have already deleted (or abandoned) any file older than ~40min. Therefore a file older than 1h that is still live should not exist under normal operation.

However, there's a subtle real edge: the **generate retry/processing waves** can stack. Let me reconsider whether a single `analyze_video` can keep a file alive beyond 1h. Worst case: processing poll = 1200s (20min). Generate: 6 attempts. But wait ? is the op_timeout per the whole `generate_content` HTTP call, and can a 503-retry-wave plus backoff push it longer? 6×180 + 93 backoff ? 1173s (~20min). Total ~2373s ? 40min. The upload retries add ~6s. So absolutely under 1h. There is no single-call path exceeding 1h.

Let me double-check there isn't a larger op_timeout or longer poll elsewhere (e.g., the gemini.py segmenter's chunk path, which might use different policies).

25. user (2026-06-25T17:01:27.422Z)

```
200
201                 # Extract chunk with ffmpeg
202                 logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
203                 success = extract_video_chunk(
204                     video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
205                     reencode=ai_scale_width > 0,
206                     scale_width=ai_scale_width if ai_scale_width > 0 else None,
207                 )
208                 if not success:
209                     logger.error(f"Failed to extract {chunk_label}. Skipping.")
210                     return chunk_label, [], {}
211
212                 # Measure chunk size
213                 try:
214                     chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
215                 except Exception:
216                     chunk_size_mb = 0.0
217
218                 # Get prompt from sport profile
219                 prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
220
221                 # Process this chunk through the provider
222                 chunk_segments, metrics = provider.analyze_video(
223                     chunk_path, prompt, chunk_duration=actual_chunk_dur, label=chunk_label
224                 )
225                 metrics["file_size_mb"] = chunk_size_mb
226                 metrics["segments_found"] = len(chunk_segments)
227
228                 # B2: surface provider failures so callers can distinguish "API error"
229                 # from "genuinely no rallies in this chunk" (both return [] today).
230                 if metrics.get("error"):
231                     logger.warning("[%s] provider error ? chunk %d skipped: %s",
232                                    chunk_label, ci, metrics["error"])
233
234                 # Offset timestamps back to full-video coordinates
235                 for segment in chunk_segments:
236                     if "start_time" in segment:
237                         try:
238                             segment["start_time"] = float(segment["start_time"]) +
chunk_start
```

```
239                         except (ValueError, TypeError):
```

26. assistant (2026-06-25T17:01:39.028Z)

All file lifecycle goes through `analyze_video`, which always deletes within the same call (~40min worst case). So the per-file lifetime is well under the 1h grace, in any process.

This significantly weakens the finding's central scenario. The finding posits process B deleting process A's "live" file at "create_time older than cutoff" ? but for a file to be both (a) older than 1h AND (b) still genuinely owned/in-use by A, A would have to hold a file beyond its own ~40min per-call ceiling, which the code does not do. The only way A still "owns" a >1h-old file is if A leaked it (hard crash) ? in which case it is genuinely orphaned and SHOULD be swept (that's the feature's purpose).

Let me verify there's no clock-skew or create_time-vs-upload-time subtlety, and confirm the docstring wording precisely. The docstring says "so we never race a concurrent worker mid-upload" and "Protects a file a live worker may still be processing." Given the per-call ceiling is ~40min and the grace is 60min, the grace DOES in fact protect any file a live worker is processing in the same-key scenario ? a worker never processes a file for >1h. So the docstring claim, while absolute-sounding, is actually backed by the timing margin (40min < 60min).

Let me confirm the upload-time math once more and check whether `ai_max_workers` concurrency could stall the ThreadPool such that an early-submitted window's file sits queued. Actually no ? `files.upload` happens inside `analyze_video` which only runs when the worker picks up the task; a queued window hasn't uploaded yet, so no file exists. Good.

Let me also confirm the test file to see what guarantees the PR author actually asserts (whether they claim cross-process safety).

27. user (2026-06-25T17:01:40.018Z)

```
1       import logging
2       from unittest.mock import MagicMock, patch
3
4       from backend.providers.gemini import GeminiProvider
5
6
7       def test_gemini_provider_generate_failure_is_flagged():
8           """A generate() give-up (per-attempt timeout / 503 burst after retries) must set
9           metrics['generate_failed'] so a FAILED chunk isn't silently mistaken for 'no
rallies'
10          (the silent-partial-result guard the segmenter surfaces loudly)."""
11          with patch("backend.providers.gemini.genai") as mock_genai, \
12               patch("backend.providers.gemini.run_bounded", side_effect=RuntimeError("503
exhausted")):
13              client_mock = MagicMock()
14              mock_genai.Client.return_value = client_mock
15              file_mock = MagicMock()
16              file_mock.state.name = "ACTIVE"
17              file_mock.name = "dummy_file"
18              client_mock.files.upload.return_value = file_mock
19              client_mock.files.get.return_value = file_mock
20
21              provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
22              segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
23
24              assert segments == []                       # no analysis for this chunk
25              assert metrics.get("generate_failed") is True   # ...but the failure is FLAGGED,
not silent
26
27
28      def test_gemini_provider_success():
29          # Mock GenAI client
```

```
        with patch("backend.providers.gemini.genai") as mock_genai:
            client_mock = MagicMock()
            mock_genai.Client.return_value = client_mock

            # Mock file upload
            file_mock = MagicMock()
            file_mock.uri = "gs://dummy"
            file_mock.name = "dummy_file"
            file_mock.state.name = "ACTIVE"

            client_mock.files.upload.return_value = file_mock
            client_mock.files.get.return_value = file_mock

            # Mock generate_content response
            response_mock = MagicMock()
            response_mock.text = '[{"start_time": 1.0, "end_time": 2.0}]'
            client_mock.models.generate_content.return_value = response_mock

            provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
            segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)

            assert len(segments) == 1
            assert segments[0]["start_time"] == 1.0
            assert metrics["upload_time"] >= 0
            assert metrics["process_time"] >= 0
            assert metrics["gen_time"] >= 0

            # Ensure delete is called
            client_mock.files.delete.assert_called_with(name="dummy_file")

    def test_gemini_provider_invalid_json():
        with patch("backend.providers.gemini.genai") as mock_genai:
            client_mock = MagicMock()
            mock_genai.Client.return_value = client_mock
            file_mock = MagicMock()
            file_mock.state.name = "ACTIVE"
            client_mock.files.upload.return_value = file_mock
            client_mock.files.get.return_value = file_mock

            response_mock = MagicMock()
            response_mock.text = '{"not_a_list": true}'
            client_mock.models.generate_content.return_value = response_mock

            provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
            segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
            assert len(segments) == 0

    def test_gemini_provider_rejects_oversize_before_upload(monkeypatch, caplog):
        with patch("backend.providers.gemini.genai") as mock_genai:
            client_mock = MagicMock()
            mock_genai.Client.return_value = client_mock
            # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
            monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
                                lambda p: 600 * 1024 * 1024)
            provider = GeminiProvider(api_key="k", model_name="gemini-flash-latest")
            with caplog.at_level(logging.ERROR, logger="backend.providers.gemini"):
                segments, metrics = provider.analyze_video("big.mp4", "p")
            assert segments == []
            client_mock.files.upload.assert_not_called()
            # The refusal must be LOUD and machine-visible: callers aggregate the metrics
            # flag into the per-video report (oversize_clip_skipped rule).
            assert metrics["oversize_skipped"] is True
```

```
92              assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
93
94
95      def test_gemini_provider_upload_fail():
96          with patch("backend.providers.gemini.genai") as mock_genai:
97              client_mock = MagicMock()
98              mock_genai.Client.return_value = client_mock
99              client_mock.files.upload.side_effect = Exception("Upload err")
100
101             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
102             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
103             assert len(segments) == 0
104
105
106
107     # --- O1/P11: Gemini remote-file GC (startup orphan sweep) ---
108
109     def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
110         f = MagicMock()
```

28. assistant (2026-06-25T17:01:53.439Z)

I have enough to evaluate. Let me synthesize the assessment.

The finding has accurate structural facts but its core causal claim is undermined by the timing math:

Accurate parts:
- The lock IS keyed to `db_path + ".lock"` (main.py:1319) ? different `--output-dir`/`RALLY_APP_HOME` run concurrently.
- `backend.worker` IS a separate entrypoint that runs the same Gemini segmenter pipeline (via the registry, `provider.analyze_video` ? `files.upload`) and never acquires the lock or calls the sweep.
- All processes share one `GEMINI_API_KEY` namespace; `files.list()` returns every file for that key. The sweep is global by mime+age, not by ownership.
- A 403/404 on a deleted file would indeed surface as `generate_failed`/`status check failed`, dropping a chunk's rallies (the segmenter flags it but rallies are lost).

The fatal flaw in the scenario: For process B's sweep to delete a file process A still legitimately owns, the file must be **older than the 1h grace** AND still in active use by A. But every file's entire lifecycle is contained within ONE `analyze_video` call, whose worst-case lifetime is bounded: processing poll `max_wait_sec=1200s` (~20min) + generate (6 attempts × `op_timeout_sec=180s` + ~93s backoff ? ~20min) ? **~40min worst case**, comfortably under the 60min grace. `_safe_delete` runs at the end of that same call on every path. So A never holds a file alive past ~40min; any file older than 1h that A "still owns" would mean A leaked it via a hard crash ? i.e., it is genuinely orphaned and SHOULD be swept (the feature's exact purpose). The finding's "see worst-case lifetime below" promises a figure it never substantiates, and the actual figure refutes the premise. The docstring's "never race a concurrent worker mid-upload" is in fact backed by the 40min < 60min margin.

The finding's suggestions (ownership tagging, opt-in flag) are reasonable hardening for a hypothetical multi-tenant future, but the described data-corruption race is not actually reachable given the bounded per-call file lifetime. This is a docstring-tone nit at most, not a MEDIUM correctness/data-loss bug.

29. user (2026-06-25T17:02:02.921Z)

Structured output provided successfully

30. assistant (2026-06-25T17:02:10.467Z)

I've completed the adversarial verification. My assessment: **isReal=false, adjustedSeverity=INVALID**.

The finding gets the structural facts right (db_path-keyed lock, `backend.worker` as a lock-bypassing separate entrypoint, shared GEMINI_API_KEY namespace, ownership-blind sweep), but its central data-corruption scenario is refuted by the timing math the finding itself fails to substantiate: every uploaded file's full lifecycle lives inside one `analyze_video()` call whose worst-case wall time (~40min: 1200s processing poll + ~1173s bounded generate-with-retries) is comfortably under the 1h grace, and `_safe_delete` always runs at the end of that call. So no live process legitimately holds a file older than 1h ? any such file is a genuine crash-orphan that the sweep is *supposed* to reclaim. At most this is a docstring-tone nit, not a MEDIUM bug.

Key files: `backend/providers/gemini.py:44-84` (sweep) and `:186` (`_safe_delete` always reached), `backend/main.py:1319` (lock keying) and `:1345` (sweep call), `backend/worker/__main__.py:158-269` (lock/sweep-bypassing entrypoint), `backend/infrastructure/execution_policy.py:48-51` (timing constants).